

Week Three Lecture Notes

Evan M Drake
drakeev@butte.edu

02/02/2025

Contents

1	Programmatic Problem Solving	2
1.1	Assess Problem	2
1.2	Define Problem	2
1.3	Gather Data	2
1.4	Develop Algorithm	3
1.5	Program Algorithm	3
1.6	Evaluate	4
1.7	Implantation of program	4
2	Files and Functions	4
2.1	File systems	5
2.2	File IO	5
2.3	Functions	6
	References	I
A	Helpful Links	II

1 Programmatic Problem Solving

Lets discuss how we should approach solving problems with programming. It is always a good idea to have an overview of the process. It is my belief that starting on paper and "mathing out" the problem is the best start.

1.1 Assess Problem

We need to understand the problem requirements. The information at our disposal and the information required for the problem. Some times this is going to mean research into the problem space. For example if you are trying to design a stop light at an intersection there will be an education period. What are the subtlety's of the problem space? The light should detect cars; how is this commonly done? What are the mechanisms that detect cars and how do we interact with them. The problem assessment will always be the place with the most research required until we have a good understanding of the *problem space*.

1.2 Define Problem

Once we understand the problem space, maybe we have a list of each sub-mechanism and how we will interact with them. We will begin to design our problem, think of this as drawing a boarder around our problem subset so we can *divide and conquer*. Think of this period as breaking our problem space up into smaller and smaller problems, then find a subset of those that we need to solve. For the traffic light problem that may mean we simply need the green light to turn on in sequence after the red light turns off. This is a much simpler task than the original "build a traffic light" problem.

1.3 Gather Data

Here we will gather data on our problem subset. Keeping with our traffic light analogy, this may mean looking up how we take signals from the individual lights. This will always mean some sort of data gathering for the mechanics of your actual solution.

1.4 Develop Algorithm

Algorithm development can happen in many forms though there are two popular ways. The **Top-Down** approach allows us to begin with the input data and work our way to producing the required output data. The **Bottom-Up** approach will begin with the required result and build an algorithm that produces that result using some specific input data. In engineering it should nearly always be the case that your input data is known at the start so the top down approach may look easier. I challenge you to look at both approaches in detail as there will be times where one is obviously superior and times when one is not. Here, using one of the above approaches we will formulate some math or pseudocode for our algorithm.

```
// pseudo code for light change
green_on = getGreenLightStatus()
red_on   = getRedLightStatus()

if green_on {
    waitTenSec()
    turnOffLight(green)
    turnOnLight(red)
}
if red_on {
    waitTenSec()
    turnOffLight(red)
    turnOnLight(green)
}
```

1.5 Program Algorithm

In this step we choose a language and begin to program. We design this program with the objective of fitting our algorithm into the context of its environment. If there is preexisting code, we must account for this and adjust our code to fit that preexisting code in style and format. All cases in this class will be that you do not have to match preexisting code unless otherwise specified. Here will choose MATLAB as the language and write some matlab code to solve our problem.

```
g = greenStatus() \% 1 == green on, 0 otherwise
```

```

2  r = redStatus()           \ % 1 == red on, 0 otherwise
4  if g == 1
        waitTenSec()
6      turnOffLight('green')
        turnOnLight('red')
8  end
    if r == 1
10         waitTenSec()
        turnOffLight('red')
12        turnOnLight('green')
    end

```

The above MATLAB code assumes that we have some predefined functions, namely: `greenStatus`, `waitTenSec`, `turnOffLight`, and `turnOnLight`. This is valid MATLAB code otherwise and will fit as the implementation of our pseudocode.

1.6 Evaluate

Now we have to test our code and ensure that it will function properly in the desired case. We will always have some way to test code, whether it is formal tests, type validation, or by hand. Testing ensures that when we use the code in the field failure is more controlled. This is out of the scope of this course.

1.7 Implantation of program

At this step we push the code to the desired hardware and it runs. This is out of the scope of this course.

2 Files and Functions

Here we will discuss files on a computer, how they work, how we will use them, and how MATLAB uses them. We then venture into the topic of functions examining how they help us and how to use them.

2.1 File systems

What is a file system? Anyone who has used a computer (even before GUI's) will understand that accessing files is a crucial role of computers. A file system is essential your computers method of keeping track of each *file* on the system. In windows this is very complex, thus here I will explain things in terms of Linux¹ Every piece of data on a computer is a file of some kind, thus keeping track of files is a priority job of the OS. The file system allows for efficient indiscriminate tracking of each and every file. The structure of the file system is something that we call a *tree* in computer science. Which looks something more like an upside down tree when drawn but non-the-less is a branching, growing structure which is quickly searchable. Each file is callable through this tree and lives in a specific and special place. As this applies to MATLAB there is only specific important info to remember.

1. Each file has a specific name
2. Each file has an extension that defines its contents
3. Each file has a special location

If for example we look at the file `/Documents/classes/csci04/teach.py`. The **location** of this file is `/Documents/classes/csci04`. Which translates into: in the `csci04` directory; which is inside the `classes` directory; which is inside the `Documents` directory; which is inside the `/` directory. The **name** of the file in question is `teach.py`. Using the name we can call on this file from anywhere in the file system as long as we know the location as well. The **extension** of this file is `.py`, signifying it as a python file containing the python programming language.

2.2 File IO

MATLAB contains many ways to import data from different kinds of files. In this course we will utilize `.csv` files for our data usually. These are standard scientific spreadsheet style data files. MATLAB reads these files most easily using the **readtable** function.

```
1 T = readtable("squid.csv")
```

¹this makes no difference to an engineering student I simply felt that I should state it here.

The above code will read in the data from file *squid.csv* into the variable *T*. If and only if the desired file is in your working directory, your working directory being the directory you are currently working in. In the MATLAB online environment you will not have to worry about the working directory.

2.3 Functions

Functions are powerful tools in mathematics, allowing for abstracting complexity and providing portability. In programming functions have the same sort of usefulness, with portability being the most useful attribute here. When a function is portable it means that invoking that function is easier than rewriting all the logic of the function. Examples of function invocations can be seen above in the traffic light code examples, they allowed me to abstract complexity of certain systems. Function going forward will provide us with easily reusable code. In MATLAB the syntax for a function is a bit esoteric.

```
1 function o = squid(x,y)
    o = x + y;
3 end
```

The above fxn has the name *squid*. The fxn takes two inputs, namely *x* and *y*, and has output *o*. We define the structure of a function definition below. Using the function once defined is quite easy, simply call it with desired inputs $\text{squid}(5,6) \triangleq 11$

```
function <output name> = <name>(<input list >)
    <function body>
    <output name> = <result >;
end
```

References

- [1] Tom Rebold Nick Langhoff Erik Dunmire. *MATLAB*. 2017. URL: <https://canadacollege.edu/nsf-iuse/curriculum.php>.

A Helpful Links

List of common commands